2025 11th International Conference on ICT for Sustainability (ICT4S) | 979-8-3315-8717-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICT4S68164.2025.00016

Container-level Energy Observability in Kubernetes Clusters

Bjorn Pijnacker
University of Groningen
Groningen, the Netherlands
b.pijnacker@rug.nl

Brian Setz
University of Groningen
Groningen, the Netherlands
b.setz@rug.nl

Vasilios Andrikopoulos
University of Groningen
Groningen, the Netherlands
v.andrikopoulos@rug.nl

***Abstract*—Kubernetes has been for a number of years the default cloud orchestrator solution across multiple application and research domains. As such, optimizing the energy efficiency of Kubernetes-deployed workloads is of primary interest towards controlling operational expenses by reducing energy consumption at data center level and allocated resources at application level. A lot of research in this direction aims on reducing the total energy usage of Kubernetes clusters without establishing an understanding of their workloads, i.e. the applications deployed on the cluster. This means that there are untapped potential improvements in energy efficiency that can be achieved through, for example, application refactoring or deployment optimization. For all these cases a prerequisite is establishing fine-grained observability down to the level of individual containers and their power draw over time. A state-of-the-art tool approved by the Cloud-Native Computing Foundation, Kepler, aims to provide this functionality, but has not been assessed for its accuracy and therefore fitness for purpose. In this work we start by developing an experimental procedure to this goal, and we conclude that the reported energy usage metrics provided by Kepler are not at a satisfactory level. As a reaction to this, we develop KubeWatt as an alternative to Kepler for specific use case scenarios, and demonstrate its higher accuracy through the same experimental procedure as we used for Kepler.**

***Index Terms*—kubernetes, kepler, energy consumption, power draw, energy observability, empirical evaluation**

I. Introduction

Datacenters and cloud computing are significant power users. In 2021, cloud computing accounted for approximately 1 % of global power usage, with it estimated to reach 8 % before 2030[1]. As the leading container orchestration platform, Kubernetes-based workloads play a significant role in this power consumption. According to a 2022 Red Hat report, up to 70 % of IT organizations use Kubernetes (K8s) in some way[2]. In this respect, the energy usage and efficiency of applications running in Kubernetes clusters is of significant interest, since significant savings can be made for both cost and carbon emissions in data center-based computing.

Existing solutions attempt to optimize Kubernetes cluster or even data center power usage, and therefore energy consumption, as a whole [1], [2]. This has yielded some promising results. However, a relatively unexplored aspect of Kubernetes energy optimization is that of targeting energy usage at workload deployment level. To achieve any yield in this effort, it becomes essential to first establish *energy observability* in K8s clusters across different levels of granularity: from complete clusters and all the way down to individual containers running in pods. Having such observability features is also an essential building block towards addressing the challenge of efficient carbon footprint measurement as discussed by Jay et al. [3], especially given that K8s clusters often contain multiple workloads from potentially different tenants.

The current state-of-the-art energy measuring tool specifically designed for this purpose is Kepler [4], [5], which at the time of writing this paper is a sandbox maturity project under the Cloud Native Computing Foundation (CNCF) umbrella[3]. While Kepler has already been used in a number of research works as a source of energy consumption data [6]–[8], beyond an initial evaluation in [4] there has been no systematic evaluation of its accuracy in the literature — at least to the extent of our knowledge. This is particularly important because as indicated by Centofanti et al. [9], there are discrepancies observed in the reported measurements under experimental conditions for this tool. This follows a trend among tools with similar purposes [3] since they rely on power modeling instead of actual measurements.

As such, this work aims to assess Kepler's fitness for purpose, and where necessary to provide alternatives and improvements. In this effort, we adopt an empirical stance, and design a replicable experimental procedure which we execute under controlled conditions. We collect and interpret the resulting data, and based on our findings we opt to develop an alternative to Kepler that does not appear to have the same issues for the same assessment.

The rest of this paper is as a result structured as follows. In the following section (Section II), we present Kepler in more depth as a background for this study; in Section III we design and present an experimental evaluation of Kepler's accuracy. In Section IV we introduce KubeWatt as an alternative approach to Kepler, which we evaluate using the same procedure as we used for Kepler in Section V. Related research is discussed in Section VI and conclusions are drawn in Section VII.

[1] https://spectrum.ieee.org/cloud-computings-coming-energy-crisis

[2] https://www.redhat.com/en/resources/state-of-enterprise-open-source-report-2022

[3] https://www.cncf.io/projects/kepler/

979-8-3315-8717-8/25/$31.00 ©2025 IEEE
DOI 10.1109/ICT4S68164.2025.00016

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

II. CNCF KEPLER

The **K**ubernetes-based **E**fficient **P**ower **L**evel **E**xporte**r** (Kepler) [4], [5] is a CNCF project that aims to estimate the power consumption of different Kubernetes components and export this data to Prometheus, a time-series database. At its core, Kepler uses the extended Berkeley Packet Filter (eBPF)[4], a technology to allow running programs in the Linux kernel to obtain energy-related system metrics. It also collects various real-time power consumption metrics using different sources, such as RAPL (Running Average Power Limit) for CPU and DRAM, NVML (NVIDIA Management Library) for GPU power, the Advanced Configuration and Power Interface (ACPI) power management interface, Integrated Dell Remote Access Controller (iDRAC)'s Redfish or Intelligent Platform Management Interface (IPMI) for platform power, or regression-based models when no real-time power metrics are available[5].

By combining utilization metrics with platform and component power usage, Kepler can estimate the power consumption of each process, container or pod. This is done by dividing the total power consumption in *idle* and *dynamic power*, using Kepler's so called *ratio power model*. Dynamic power is directly related to resource utilization and therefore attributed to the process it is responsible for the resource usage. The idle power of the host is then distributed among processes in accordance with their size, as is stipulated in the GreenHouse Gas protocol[6] guidelines.

A distinction is made between node metrics and container metrics. The node metrics are collected for each Kubernetes node and are split into core, DRAM, package, platform, and uncore components. The power data source that is used determines how exactly the power consumption of each component is derived, and multiple data sources are possible. For example, when using Redfish, the platform power is taken directly from Redfish while the other components are derived from RAPL. The container metrics on the other hand are collected for each container that is running on the Kubernetes node, and these include the same components that are collected on node level.

Kepler can be deployed in two different configurations depending on the metrics that are available in the host environment. In its most basic deployment Kepler estimates the power consumption using utilization metrics of the host and feeding this data to a pre-trained power estimation model. It is also possible to train a custom power estimation model. The second deployment mode does not use a trained model, but instead retrieves power metrics directly from the bare-metal host using one of the aforementioned power metric sources. Kepler can also use a combination of the two configurations if some bare-metal metrics are missing, to estimate these missing components. A third deployment mode is hypothesized but not currently available. This mode enables Kepler to perform its calculations inside virtual machines. This requires Kepler to be deployed on the bare-metal host as well as on the virtual machine, monitoring the idle and dynamic power of each VM using the bare-metal metrics, and then expose this power data to the Kepler instance running inside the VM itself[5].

While Kepler is applied in several academic works, its accuracy and fitness for purpose have not, and to the extent of our knowledge, been evaluated yet. Gudepu et al. [6] use Kepler to obtain power measurements and indicate it produces similar results to Scaphandre, a tool which predicts power usage per resource. However, neither tool is validated using a source of ground truth power consumption. Additionally, the Kepler results are not published, it is only mentioned that the results are similar to Scaphandre. Soldani et al. use Kepler as a demonstration of an eBPF use-case. While they show a dashboard of energy measurements, these are not validated as being accurate [7]. Centofanti et al. [9] investigate Kepler in comparison to Scaphandre and s-tui, a graphical stress-test and CPU monitoring tool[7]. However, for Kepler they only consider a configuration using the pre-trained power estimation model. They find that there are significant discrepancies between the tools used in their tests and conclude that further research is required to increase the robustness of these tools. Andringa [10] also investigates Kepler, concluding that Kepler's metrics are not accurate where total cluster power is concerned, but the discussed evaluation lacks the necessary depth and the per-container power attribution is not investigated further.

It is clear that these previous works are not sufficient to validate Kepler's accuracy and fitness for purpose. To evaluate these properties, we must first validate that the power reported per container by Kepler matches closely to what is expected taking into account resource usage by that workload. Power draw measurements are fundamental towards calculating energy consumption at this level for each workload.

III. KEPLER EVALUATION

We are interested in evaluating Kepler in terms of its main goal: container power attribution in Kubernetes clusters over time. To this end, the following research question is asked:

RQ1 How well does Kepler attribute power usage to containers on a Kubernetes node?

In order to answer this question empirically we design an experimental procedure that we discuss in the following, starting from the system under test.

A. System under Test

Before Kepler's accuracy can be evaluated through experimentation, the system under test and its constituent software and hardware stacks need to be clearly defined. The critical components of the setup include a Kubernetes cluster to run workloads in, an observability stack to collect node and container level metrics, and a way to measure the ground-truth

[4]https://ebpf.io/what-is-ebpf/

[5]https://www.cncf.io/blog/2023/10/11/exploring-keplers-potentials-unveiling-cloud-application-power-consumption/

[6]https://ghgprotocol.org/

[7]https://github.com/amanusk/s-tui

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Metrics collected and used for the experiments

Source	Metric	Explanation
iDRAC	`power_control_avg_consumed_watts`	Average power consumption in W over the last measured interval (1 min)
cAdvisor	`container_cpu_usage_seconds_total`	Cumulative CPU time consumed in seconds
Kepler	`container_joules_total`	Aggregated platform and component power per container in J
	`container_cpu_instructions_total`	Number of CPU instructions measured per container

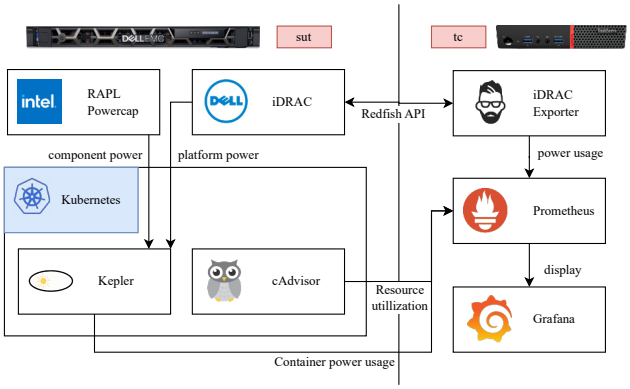


Fig. 1: The hardware and software setup to evaluate Kepler. `sut` = system under test, `tc` = thin client.

power consumption of the cluster. The complete setup for this purpose is shown in Fig. 1.

The hardware setup that is used consists of two machines: a System Under Test (SUT), and a Thin Client (TC) for data collection, analysis, and visualization. The SUT is a Dell PowerEdge R640 server. It is equipped with two Intel Xeon Gold 6226R processors totaling 32 cores and 64 threads, 96 GB of RAM and 256 GB of RAID1 SSD storage. Fedora Server 40 is installed as the operating system. The server is also equipped with Dell iDRAC9 from which we can obtain, among others, the ground-truth power metrics using the Redfish API integration. We verified that the Redfish API data is indeed the ground-truth by including an external power monitoring wall plug. The TC is a Lenovo ThinkCentre M910q with an Intel Core i3-6100T, 8 GB of RAM, and 256 GB of NVMe storage running Fedora Server 40 as the operating system. This machine is used to collect and analyze the data from the tests we perform on the SUT so that they do not influence test results of the SUT.

A single-node Kubernetes cluster is deployed on the SUT. This cluster is bootstrapped using Rancher Kubernetes Engine (RKE)[8] version 1.5.8. RKE allows us to administer the cluster without running specific software on-node, as it uses SSH to set up the cluster on the node. For the purposes of the experiments, the exact Kubernetes distribution does not matter as long as it can be installed on bare-metal Linux. The cluster is set up in its most basic form, and we remove any workloads that are not necessary for our tests such as `nginx-ingress-controller` in order to reduce the amount of noise in our power and CPU metrics.

The backbone of the observability stack that is used to collect various metrics from the SUT consists of Prometheus and Grafana. Prometheus is the monitoring system and time series database, and Grafana is used for visualization. All components that produce metrics that require collecting support collection by Prometheus, and as Prometheus has CNCF graduated maturity, it is the recommended choice for this use case[9]. Both Prometheus and Grafana are deployed and configured as services on the TC, so the data processing does not affect the power usage of the SUT.

The metrics of the SUT that are collected by Prometheus are scraped from three different endpoints provided by: iDRAC Exporter, cAdvisor, and Kepler. The iDRAC Exporter[10] exposes the iDRAC metrics of the SUT. This exporter runs on the TC and interfaces with iDRAC's Redfish API to collect the required metrics. Whenever the iDRAC exporter is scraped by Prometheus, it uses the Redfish API to collect the data and converts it to the format expected by Prometheus. The power supply metrics collected from this endpoint are of particular interest, as they form the ground-truth power usage of the SUT. The node's Kubernetes metrics are made available for scraping by cAdvisor[11]. This allows us to monitor the per-container metrics such as CPU and memory usage. Kepler additionaly exposes an endpoint for metric scraping, by default.

Finally, Kepler is deployed in the Kubernetes cluster on the SUT. Specifically, Kepler version 0.7.2 is deployed, as newer versions suffer from an issue where incorrect values are sometimes measured at random[12]. Kepler is configured to use the Redfish iDRAC integration for platform power. Since Redfish cannot provide component power, RAPL is used instead. This configuration ensures that Kepler has access to the most accurate power source, as iDRAC can measure the server power supplies directly. A list of the metrics from each source that are used for the experiments is available in Table I. The Helm chart values for deploying the cluster and the observability stack are available together with the presented experimental data below in the replication package for this work[13].

[8] https://github.com/rancher/rke

[9] https://www.cncf.io/projects/prometheus/

[10] https://github.com/mrlhansen/idrac_exporter

[11] https://github.com/google/cadvisor

[12] https://github.com/sustainable-computing-io/kepler/issues/1344

[13] https://doi.org/10.5281/zenodo.14332659

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

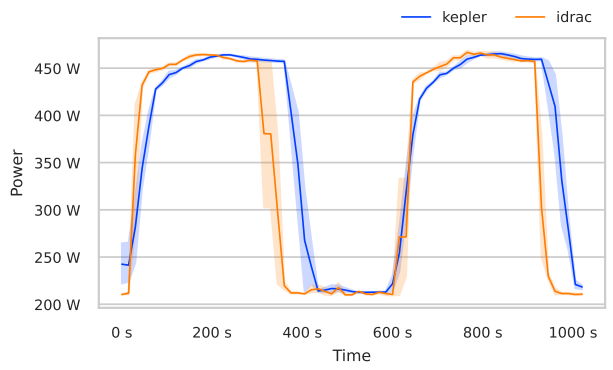


Fig. 2: Total power of SUT during the stressor tests

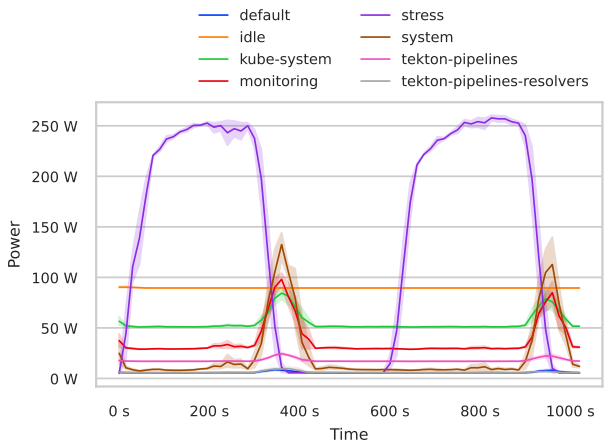


Fig. 3: Per-namespace container power draw as reported by Kepler during the stressor tests

B. Experiment Design

To evaluate Kepler's container power attribution we run a simple stress workload on the test cluster, and observe Kepler's container power metrics. While running the test we investigate not only the measurements that Kepler gives for our running test container but also look at the power draw/energy usage of the other containers in the cluster. The stress workload on our Kubernetes cluster is invoked using the Linux comman `stress-ng -cpu 32 -timeout 5m`. Afterwards, we let the system sleep for 5 minutes before running the test load again. This sequence of fifteen minutes is repeated 3 times. We expect to clearly see the generated load as power usage for our test container while other containers remain stable in their power usage throughout.

Before running these tests, we also set up 16 idle containers. They will simply run the `date` command once before we start taking measurements and then exit. These containers will therefore have the 'completed' status while our main workload is running. As such, they should not interfere with the dynamic power attribution by actually using power. Deploying these containers should also yield a situation closer to a real-world cluster, where a container of interest is not isolated on the cluster. However, as we do not want these containers to influence CPU utilization and power usage, they remain idle. We therefore expect these containers to not be attributed any power during the runtime of our tests.

C. Results

Figures 2 and 3 visualize the test results for our Kepler stressor test. In Fig. 2, the total power for SUT as measured by both iDRAC and Kepler is indicated. Visually, the measurements align quite closely. Between iDRAC and Kepler there is a root mean square error (RMSE) of $66.4\,\mathrm{W}$. This rather large error is mostly caused by differences in reporting latency between iDRAC and Kepler, as also evident in the figure. Considering the total energy usage during the test instead of the wattage over time, we see an error of less than $1\,\%$, which is more than acceptable.

Figure 3 shows the attributed container power of Kepler summed by Kubernetes namespace. The results are partially as expected: the power attributed to the 'stress' namespace, which houses exclusively the stress-test container running our workload, corresponds to the generated load which was expected. Two things in this figure are not as expected however:

1) there is a peak of power usage in other namespaces after the stress-test stops, and
2) the 'idle' namespace consistently uses approximately $100\,\mathrm{W}$, even though it should not be using any energy.

The peak can be explained by considering that iDRAC may measure power usage slower than Kepler measures CPU usage. We verify this in Fig. 4, where we indeed see that power consumption metrics lag behind CPU load metrics by up to $1\,\mathrm{min}$ when CPU load quickly decreases. This finding is inline with the specification provided by iDRAC and Redfish, as this specifies power metrics[14] are updated on a one-minute interval. In this situation, where not all metrics are updated at the same interval, Kepler needs to attribute more power usage among containers than is actually occurring at that given moment, therefore artificially spiking the power usage of all workloads as CPU usage suddenly decreases.

We additionally see unexpected power usage of the 'idle' namespace, with all containers in the 'idle' namespace as reported to be using around $6.25\,\mathrm{W}$ each. Recall that we created these containers as part of the test, and that the containers are not running. All power attribution to these containers is of the idle power mode, where the power attributed to our stress container was dynamic. This means that Kepler is effectively reporting power usage for containers that are not actually running. While there is indeed overhead to managing non-running containers in Kubernetes, idle-mode power should not be equally divided among them, as this may lead to

[14] Available through the redfish/v1/Chassis/System.Embedded.1/Power/PowerControl API endpoint.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

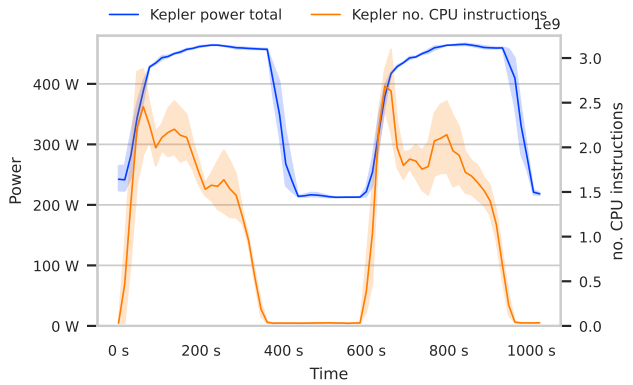


Fig. 4: Power usage and number of CPU instructions as reported by Kepler during the stressor tests

unfair attribution when the number of containers on the cluster changes. Since the total power usage of all containers was correct with respect to the iDRAC measurement, this also indicates that Kepler is at the same time likely under-reporting the idle power used by the other containers to begin with.

To see whether Kepler can correctly attribute power once the inactive containers have been deleted, we perform the following test. First, we start 64 idle containers that run `date` and exit as before. We choose a large number of idle containers so that their presence and absence has a large and thus easy to observe effect on measurements. After these pods are created and complete we give the system one minute to stabilize. We then run a small (8 CPU) stressor and delete all idle containers after two minutes, then observe how the container attribution of Kepler changes. We expect that Kepler reallocates the idle power usage to all other containers, and that the dynamic power attribution does not change.

The results of this test are presented in Fig. 5. The test was repeated four times. Figure 5a shows the power attributed by Kepler to each namespace. Figures 5b and 5c show this for idle- and dynamic-mode power respectively. Recall that the 'stress' namespace contains solely our stressor pod and that the 'idle' namespace only has the 64 inactive pods. As the test starts, the total power goes up for the 'stress' namespace as expected. As the idle pods are deleted, the idle-mode power for the 'idle' namespace quickly goes to zero as expected. Power is re-attributed throughout the other namespaces over all remaining containers.

After deleting the idle pods we also see the dynamic power usage for the 'stress' namespace going down and the dynamic power usage for the 'system' one go up. Note that the CPU usage of the workload remained at 100%, and consistent throughput was indicated throughout the experiment as per the stressor logging, as also indicated in Fig. 5d. The change in power attribution here is unexpected, since there is no 'system' namespace with running workloads. The upward trend of this namespace's power attribution matches a downward trend in reported dynamic power usage for the 'stress' namespace. According to Kepler, this power is being attributed to pods named 'system_processes', which is a reserved name in Kepler for processes that cannot be attributed to a pod. After stopping the testing workload running in the 'stress' namespace, we see that the 'system' namespace also reports using less power.

D. Interpretation

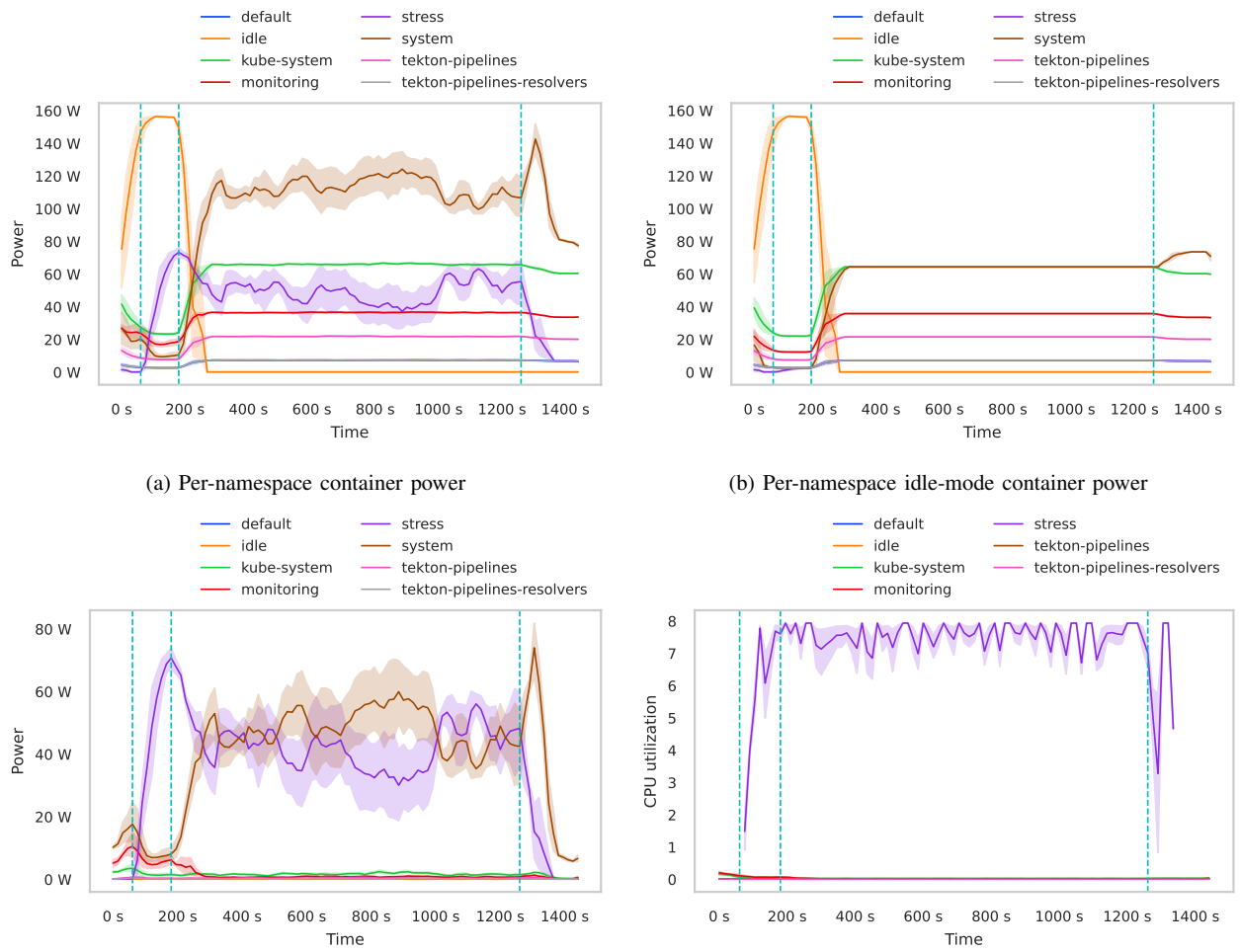
The container power attribution in Kepler leaves much to be desired. In our experiments, we have seen multiple examples of container power attribution that were clearly inaccurate and sometimes inexplicable, both in idle- and dynamic-power mode. As we have seen in the inactive-container deletion test, Kepler misattributes some power to system processes, where it should not be doing so. Additionally, as we have seen in the single-stressor test, Kepler attributes power to non-running containers, which is also not correct. Consequently, and as a response to RQ1, *Kepler does not seem to produce a trustworty measure of container energy usage in Kubernetes clusters*. The scripts to run and the resulting datasets from these experiments are available in the replication package for this work.

IV. KubeWatt

In the previous section, we have shown that Kepler is not a suitable tool for producing container-level power metrics in Kubernetes with sufficient accuracy. As an alternative, we create our own tool: KubeWatt. It will be based partially on the power attribution model which is proposed in [10], since the latter yielded promising results in container/pod power attribution.

Functionally, KubeWatt can read CPU utilization metrics from Kubernetes, obtain node power usage using a Redfish API integration, can split node power usage into static and dynamic parts and can produce Prometheus-style metrics for container power usage. The first two are obvious: in order to calculate power usage of a container, KubeWatt must at least know the total power usage of the node that container is running on, and it must know the amount of resources that container uses on the node. Dividing the power usage into 'static' (or 'idle' in Kepler) and 'dynamic' parts aims to account for the power usage of a Kubernetes cluster and server when no workloads are running. The simple act of turning on a server and running a Kubernetes cluster uses some amount of power which cannot be attributed to any specific container. KubeWatt therefore splits the total power into two components, such that the static power can be indicated in total, and the dynamic power, which is the difference between static and total power, can be attributed amongst Kubernetes containers. KubeWatt will indicate static power as a single number in its output, since it indicates overhead that is not easily attributed to any specific container. Note that this definition of overhead includes Kubernetes control plane containers. These are therefore excluded from the dynamic power attribution. Any additional power usage incurred by the control plane will instead be attributed to other running containers causing the control plane to use power.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.



(a) Per-namespace container power

(b) Per-namespace idle-mode container power

(c) Per-namespace dynamic-mode container power

(d) Per-namespace CPU utilization as reported by cAdvisor

Fig. 5: Power attribution and CPU utilization before and after deleting inactive pods. The markers indicate the times at which (1) the stressing load was started; (2) the inactive pods were deleted; (3) the stressing load was stopped, respectively.

A. Allocation Model

KubeWatt builds on the allocation model proposed in [10] to attribute total power among containers in Kubernetes, named 'pod mapping'. We do, however, make a few changes to fit the model to our purpose. More specifically, we define $\text{power}(\cdot)$ as the power usage of some component and $\text{cpu}(\cdot)$ as the CPU utilization of some component. We define $\text{power}_d(\cdot)$ and $\text{power}_s(\cdot)$ as the dynamic and static fractions of power as described above, respectively, with

$$\text{power}(\cdot) = \text{power}_d(\cdot) + \text{power}_s(\cdot).$$

Then let n_i be some Kubernetes node whose power $\text{power}_d(n_i)$ and $\text{power}_s(n_i)$ are known. Let $c_{m,i}$ be a Kubernetes container running on node n_i, and identified uniquely by m. Given the CPU utilization of $c_{m,i}$, we have

$$\text{power}(c_{m,i}) = \text{power}_d(n_i) \cdot \frac{\text{cpu}(c_{m,i})}{\sum_i \text{cpu}(c_{m,i})}. \tag{1}$$

Importantly, we make a distinction between $\sum_i \text{cpu}(c_{m,i})$, the CPU usage of all containers on a node combined, and $\text{cpu}(n_i)$ the CPU utilization of the node, as the metrics API of Kubernetes includes overhead CPU usage such as system processes in the latter metric, which is already included in static power and should not be attributed to Kubernetes containers[15]. Equation (1) gives us a metric of power usage for each container in a Kubernetes node as derived from the node power usage and container CPU utilization.

B. Architecture

KubeWatt consists of several components which interface with other applications. In this section, we discuss the *power collector* and *Kubernetes metrics collector* components. A high-level diagram of KubeWatt in context is shown in Fig. 6. More specifically:

[15]https://kubernetes.io/docs/reference/external-api/metrics.v1beta1/

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

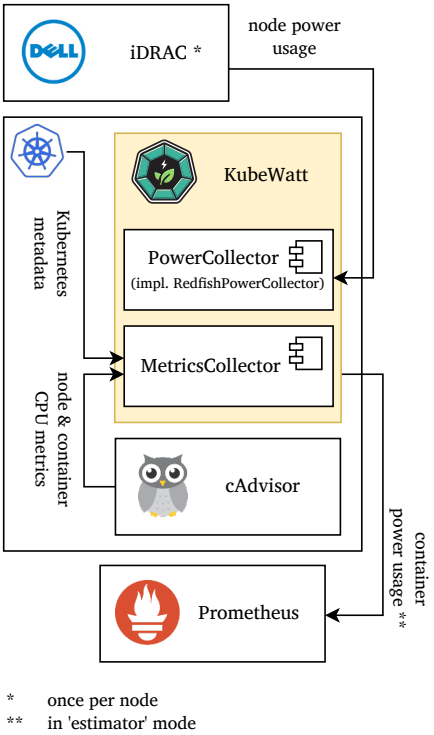


Fig. 6: High-level diagram of KubeWatt architecture

Power collector: The power collector component of KubeWatt is responsible for providing a measure of power usage in W per node in the Kubernetes cluster. KubeWatt does not care about the source of power and the implementation is abstracted behind the `PowerCollector` interface, meaning that it is easily extended to use other power sources. For the purposes of this work, the only implemented version of this interface is the `RedfishPowerCollector` class, which uses the Redfish API in SUT to obtain power usage from the power supply.

Kubernetes metrics collector: The Kubernetes metrics collection component is responsible for obtaining CPU usage metrics of both nodes and pods in the Kubernetes cluster. It uses the `metrics.k8s.io/v1beta1/pods` and `metrics.k8s.io/v1beta1/nodes` Kubernetes API endpoints for pods and nodes metrics respectively. The nodes endpoint returns CPU utilization in ns for each node. The pods endpoint returns CPU utilization in ns for each container in each pod[16]. In the Kubernetes Java API implementation, which KubeWatt uses, the pods metrics are available per namespace[17].

C. Operational modes

KubeWatt runs in one of three modes. The first two modes, 'base initialization' and 'bootstrap initialization' are initialization modes. They run as a one-off job to initialize KubeWatt parameters. These modes analyze the cluster which KubeWatt will run on and find the static power value ($power_s$) for each node in the cluster. This value is calculated once and not updated without manual intervention; therefore, we assume that this does not change over time.

1) Initialization modes: The *base initialization mode* is the simplest mode that KubeWatt can run in. It expects an empty Kubernetes cluster running no more than the Kubernetes control plane. KubeWatt expects its user to specify which pod names are part of the control plane as a set of regular expressions, such that it can validate the cluster is indeed empty before starting. Over a default period of 5 minutes, KubeWatt measures the power usage per node every 15 seconds, which is averaged to produce the static power value per node. We expect that this mode will produce the most accurate results for the static power value, as it directly measures the idle cluster. Both the period and the measurement interval can be adjusted to the needs of the specific case through the configuration file for KubeWatt; for the purposes of our experimental evaluation these values were deemed sufficient.

The *bootstrap initialization mode* is an alternative to the base initialization mode. It attempts to find the static power value from a cluster which is already running workloads that cannot be turned off for testing. Since this mode makes an estimation of the static power value based on measurements, the base initialization mode should be preferred if possible. This mode gathers both CPU-usage data for each node and power usage data for each cluster node. Data are gathered every fifteen seconds for half an hour. Afterwards, KubeWatt checks whether the data have enough variability and a sufficient distribution to draw conclusions from. If not, data collection is repeated, otherwise KubeWatt proceeds with data analysis. To gauge whether data are sufficient to perform analysis with, KubeWatt checks the rough distribution of the data. The collected CPU-usage values are placed in buckets. KubeWatt then checks the amount of measurements in the largest bucket, and validates that no bucket has fewer values than some predefined factor of it. By default, each bucket should have at least half as many values as the largest, to ensure a relatively uniform distribution. The buckets are 10 % in size between 20 % and 80 % CPU utilization by default. All of these values are configurable, but in our experience these bounds are more than sufficient for a cluster with a fluctuating, non intense over time workload.

As previously discussed, the static power should include the Kubernetes control plane utilization at idle. To achieve this, the CPU-usage of the control plane containers is gathered at the same time as the node CPU-usage and power usage. The set of control plane containers is known as the user is required to specify these when configuring KubeWatt. The control plane CPU utilization is averaged to give an indication of stable control plane CPU-usage at idle. Note that we cannot expect the control plane CPU utilization to remain stable when the cluster has load. As the idle control plane utilization is encapsulated in the static power value, any power usage caused by higher utilization in result of cluster load will be

[16]https://kubernetes.io/docs/reference/external-api/metrics.v1beta1/
[17]https://github.com/kubernetes-client/java

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

attributed among the containers causing this load. To finally derive the static power usage, a linear regression is performed on the collected data below $50\,\%$ CPU utilization, as we know from [11] that CPU utilization against power usage grows linearly for those values. The regression coefficients are subsequently used to find the estimated power usage at the average CPU-usage of the control plane. This then gives us the static power for each node in the Kubernetes cluster. As above, this utilization threshold can be defined explicitly in the configuration file for KubeWatt if necessary.

2) Estimation mode: The estimation mode is what we consider the 'main' mode of KubeWatt. This mode takes the output from either initialization mode as input and actually estimates the amount of power that each container in the Kubernetes cluster uses. When running in this mode, KubeWatt exports Prometheus-style metrics. The estimator mode uses the allocation model as described in Section IV-A.

D. Deployment

KubeWatt can be deployed in a Kubernetes cluster using the provided Helm chart[18]. There are two important configuration options that need to be set, the Redfish configuration mapping and the operation mode. It is important that the Redfish REST interface is accessible from the node on which KubeWatt is deployed. The exact configuration required to setup Redfish is dependent on the server manufacturers out-of-band management software. It is important to ensure that KubeWatt is configured to first run in one of the two initialization modes `INIT_BASE` or `INIT_BOOTSTRAP`. The base initialization mode should only be used on an empty cluster, whereas the bootstrap initialization mode should be used on an already provisioned cluster. The initialization modes will output the static power values per node. This static power value should then be configured in the value file, after which KubeWatt can be run in the `ESTIMATOR` mode to begin estimating the power usage per Kubernetes container.

V. KubeWatt Evaluation

In the previous section, we have outlined how KubeWatt works. In this section, we perform an evaluation on KubeWatt in order to validate its workings and to showcase whether or not we have improved on Kepler's limitations as discussed in Section III. For this purpose we pose three research questions:

RQ2 How accurately can KubeWatt's base initialization mode report the static power value?
RQ3 How accurately can KubeWatt's bootstrap initialization mode estimate the static power value?
RQ4 How (well) does KubeWatt attribute power usage to containers on a Kubernetes node?

The testing setup as discussed in Figure 1 is reused as-is; however, the Kepler deployment has been removed as this is no longer necessary. KubeWatt is now deployed in its place.

[18]https://github.com/bjornpijnacker/kubewatt

For the initialization modes, we want to verify that the output that KubeWatt gives is equal to the static power of the system. The static power value has been determined from iDRAC data at $199.1\,\mathrm{W}$. To evaluate the base initialization mode (RQ2), we run the KubeWatt job on SUT with nothing else running on the cluster. The following list of pod names is provided to KubeWatt as control plane: `nfs-.*`, `calico-.*`, `canal-.*`, `coredns-.*`, `metrics-.*`, `tekton-.*`, `kubewatt-.*`. Note that `tekton-.*` is not technically part of the control plane; however, it could not be easily removed and as it is idle, it should not have a significant impact on findings. We repeat the job six times in sequence, then verify that the output value closely corresponds to the expected power value. To obtain the expected power value, the power use reported by iDRAC is tracked during the runtime of each of the initialization job runs. For the base initialization mode we evaluate whether the resulting values are consistent over multiple runs and whether the resulting values are accurate to the expected value taken from iDRAC.

To evaluate the bootstrap evaluation (RQ3), we run a best-case test, where the cluster is stressed with a random stressor. Stress-ng is used to create stressors at a random CPU level between 1–64 that last three minutes each. This creates a CPU load that should be uniformly distributed across the entire CPU range of SUT. This test is repeated three times, again ensuring consistency and accuracy. To evaluate the estimation mode we repeat the test we ran for Kepler in which we deleted inactive pods while running a stressor. This test is meant to validate both that KubeWatt can accurately gauge container power usage as well as showcase that it does not suffer from the same limitations as Kepler. The replication package for this work contains all the necessary scripts and datasets, as discussed in Section III.

A. Results

1) Base initialization mode: Running the base initialization on our empty cluster, KubeWatt reports static power values of $198.9\,\mathrm{W}$, $199.15\,\mathrm{W}$, $199.1\,\mathrm{W}$, $199.1\,\mathrm{W}$, $198.75\,\mathrm{W}$, and $199.15\,\mathrm{W}$ across the six tests. These values are within $0.35\,\mathrm{W}$ of each other and within $0.3\,\mathrm{W}$ of the expected value, or in other words within an error of less than $0.2\,\%$, indicating a consistent and accurate output.

2) Bootstrap initialization mode: The measurements that KubeWatt has taken for the bootstrap initialization mode are shown in Fig. 7 for each of the three repeated tests. The raw values are shown in blue, while the resulting regression that KubeWatt performs is shown in orange. Recall that KubeWatt only performs the regression on the lower $50\,\%$ of the data. KubeWatt reports static power values of $198.44\,\mathrm{W}$, $199.58\,\mathrm{W}$, and $199.41\,\mathrm{W}$, again indicating consistent and accurate output. For each of the tests, the control plane utilization contributed approximately $0.30\,\mathrm{W}$. It is noteworthy that these tests, and the bootstrap initialization mode in general, take a lot longer to run than the base initialization mode. The three tests as shown in Fig. 7 took, respectively, $3.5\,\mathrm{h}$, $4\,\mathrm{h}$ and $13\,\mathrm{h}$ to finish.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

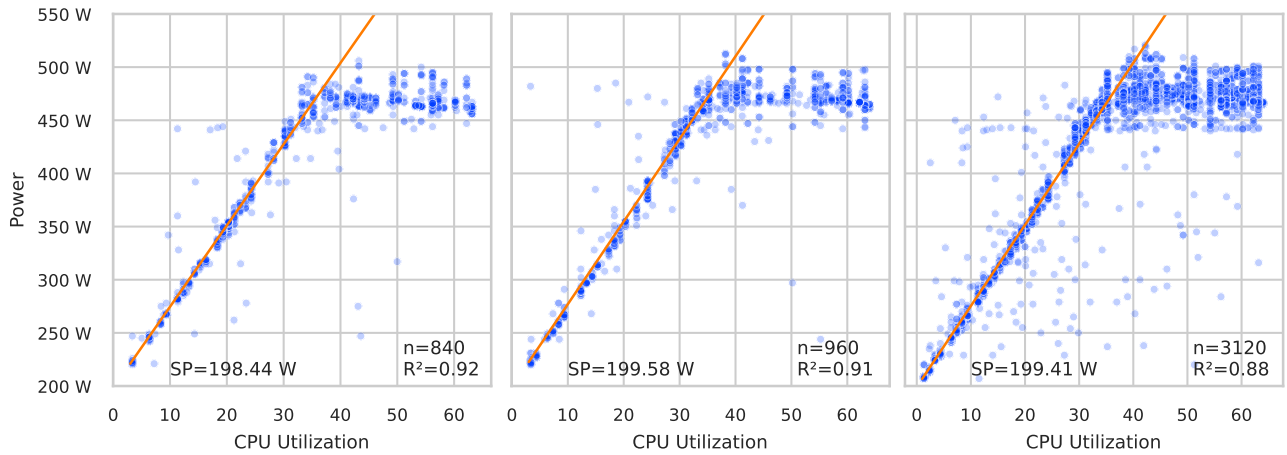


Fig. 7: CPU utilization and power usage (blue) as reported by KubeWatt for the bootstrap initialization mode. Result of linear regression on bottom half of data (orange). Repeated three times.

3) Estimation mode: The results of the repeated 'deleting inactive pods' test is shown in Fig. 8. Figure 8a shows the per-namespace container power. Note that 'static' is not a namespace, but is the namespace-less static power. Some namespaces here are missing compared to Fig. 5; this is because all containers in those namespaces are part of the control plane and are not reported separately by KubeWatt. Note also that the idle containers are not reported. This is the case because they are not using power: they are not running. Figure 8b shows the per-namespace CPU utilization. We see the same measurements as in Fig. 5d, indicating the tests were performed equally.

The main, and expected result is depicted in Fig. 8a. We see that the power usage of the stressor pod goes up with CPU utilization, stays consistent throughout the test and goes down when the stressor is stopped. Conclusively, we see from these figures that KubeWatt can accurately report the power usage of the stress pod, even when a large number of inactive pods clutter the cluster.

B. Discussion

With the results of KubeWatt experiments that have been outlined, we can now turn again to the questions posed at the beginning of this section. The base initialization mode is able to very accurately read the static power value (RQ2). In our test we have seen the base initialization mode get very consistent measurements over time, with the largest internal difference being $0.3\,\mathrm{W}$ out of a $199.1\,\mathrm{W}$ mean over six KubeWatt runs. Additionally, we have seen that the bootstrap initialization mode can also accurately estimate the static power value (RQ3). In this case, KubeWatt can estimate the static power value within $0.7\,\mathrm{W}$ of the expected value in a best-case scenario. In terms of container power attribution (RQ4), we see that KubeWatt is able to accurately portray the power used by a container based on CPU load, in line with what is expected based on CPU utilization, while being consistent with the total node power draw ground-truth measurements.

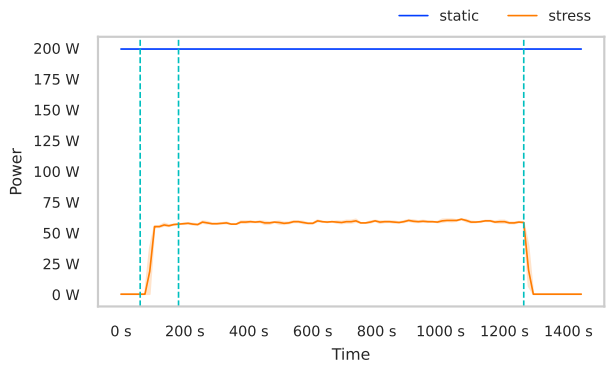
C. Limitations

There are some obvious limitations to the evaluation of KubeWatt, effectively constituting threats to the validity of this procedure, and we discuss them in the following.

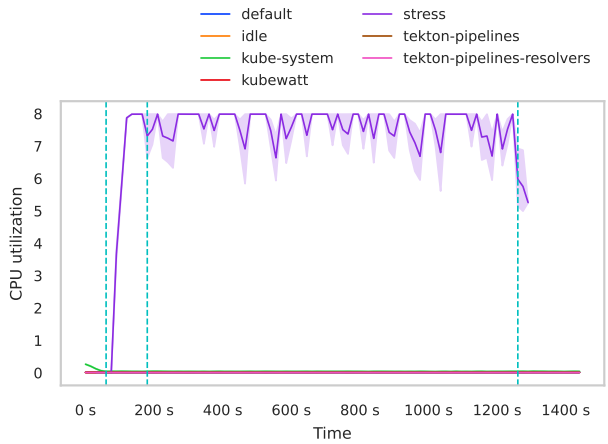
First, the SUT comprised a single server only. Evaluation results would have been more robust by combining a number of servers and distributing multiple workloads among them to more closely resemble a real-world cloud environment. KubeWatt has been designed and implemented with the general case in mind, but lack of access to multiple server devices and of time to conduct a full blown experiment prohibited us from demonstrating this; it is left as future work. Furthermore, during testing, we only ran artificial workloads using `stress-ng`. While these stress the CPU, they do so in a very specific way which may not always be equal to a real-world workload. This means that the shown results may not be completely indicative of real-world behavior. However, we have no reason to believe that the accuracy of our results would not replicate also in other applications since we make no assumption concerning the workload itself in KubeWatt.

Last but not least, our KubeWatt implementation has only been evaluated in what is for all practical purposes a private cloud deployment. Moving, for example, to a public cloud deployment would mean that we do not have access anymore to iDRAC (or equivalent) data through the Redfish API and we would also have to rely as Kepler does in this case on either RAPL data where available, or on an estimator model. Demonstrating that KubeWatt can produce consistent, if not accurate results in this scenario is left as future work. For the purposes of this experimental procedure, however, both Kepler and KubeWatt use iDRAC, which allows them to be compared reliably. An important point concerning also the evaluation of Kepler as discussed in Section III was not following the

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.



(a) Per-namespace container power



(b) Per-namespace CPU utilization as reported by cAdvisor

Fig. 8: Power attribution reported by KubeWatt and CPU utilization when running 'deleting inactive pods' test. The markers indicate the times at which (1) the stressing load was started; (2) the inactive pods were deleted; (3) the stressing load was stopped, respectively.

option of developing a custom power model for the latter. Our experience in attempting to do so was frankly frustrating to disappointing, with large gaps left in the documentation of the task that we were not able to cover on our own. Being able to build a custom power model could have perhaps resulted into more favorable for Kepler results.

VI. Related Work

Existing approaches in the literature have looked into providing observability for cloud-native applications but outside of Kubernetes and Kepler. Fieni et al. [12], for example, introduce SmartWatts, a tool for estimating container energy consumption. It works by considering component energy and hardware performance counter events to gauge resource utilization. Their model then maps power usage into static and non-static power, and divides this over the resource utilization. SmartWatts uses Linux Cgroups to allow a wide range of monitoring granularities, such as Docker containers, Virtual Machines or processes. In a follow up work, the authors propose SelfWatts [13], as a novel method to estimate component power utilization.

Dinga et al. [14] examine the performance overhead of monitoring solutions on Docker-based systems. The monitoring applications under test were ELK Stack, Netdata, Prometheus and Zipkin. They showcase a significant effect on power consumption for several of the tools and scenarios, with a $1.47\,\%$ to $12.86\,\%$ increase in energy consumption depending on the tool. They furthermore show a significant impact on CPU usage for the ELK Stack and Zipkin, and in RAM usage for all tools except Netdata. Santos et al. [15] investigate the impact of running an application in Docker versus running it on bare-metal Linux. A power meter is used to track the total system power usage when running the tests. The authors find that having the Docker service `dockerd` running the background uses a significant amount of power above bare-metal idle. In contrast to those works, our approach focuses on workloads specifically deployed in Kubernetes clusters. On the other hand, works that do focus on Kubernetes, such as those discussed in Section II, lack the necessary granularity and depth in their assessment — a gap that this work bridges as discussed in the previous sections.

VII. Conclusions

Through a set of controlled experiments, in the previous sections we have shown that the state-of-the-art energy measuring tool for Kubernetes clusters, Kepler, is able to produce accurate results at cluster- but not individual-container levels. These inaccuracies appear to be fundamental to the power allocation model used by Kepler, and can result into serious misalignment if used e.g. to attribute carbon footprint among different workloads (applications) running on a cluster. As a response to this issue, we developed KubeWatt and demonstrated that for the same scenarios it can measure power draw at both node and container levels within statistical error margins. More importantly, the power allocation model built in KubeWatt appears to produce more internally consistent results than the one used by Kepler.

Future work aims to address the limitations identified in Section V, namely, expanding the experimental evaluation in the generic case of a heterogeneous Kubernetes cluster using a set of real-world application workloads. Furthermore, by adding carbon intensity stream data to KubeWatt, we can also offer carbon footprint measurements at different granularity levels to complement the energy measurement ones.

Acknowledgements

This work was supported by ITEA4 and RVO under grant agreement No. 22035 MAST (https://itea4.org/project/mast.html). Moreover, we would like to thank Wim Nap for his help with securing the necessary infrastructure for this project.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] R. Douhara, Y.-F. Hsu, T. Yoshihisa, K. Matsuda, and M. Matsuoka, "Kubernetes-based Workload Allocation Optimizer for Minimizing Power Consumption of Computing System with Neural Network," in *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*, Dec. 2020, pp. 1269–1275. [Online]. Available: https://ieeexplore.ieee.org/document/9458062

[2] S. Ghafouri, S. Abdipoor, and J. Doyle, "Smart-Kube: Energy-Aware and Fair Kubernetes Job Scheduler Using Deep Reinforcement Learning," in *2023 IEEE 8th International Conference on Smart Cloud (SmartCloud)*, Sep. 2023, pp. 154–163. [Online]. Available: https://ieeexplore.ieee.org/document/10349157

[3] M. Jay, V. Ostapenco, L. Lefevre, D. Trystram, A.-C. Orgerie, and B. Fichel, "An experimental comparison of software-based power meters: focus on cpu and gpu," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023, pp. 106–118.

[4] M. Amaral, H. Chen, T. Chiba, R. Nakazawa, S. Choochotkaew, E. K. Lee, and T. Eilam, "Kepler: A Framework to Calculate the Energy Consumption of Containerized Applications," in *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, Jul. 2023, pp. 69–71, iSSN: 2159-6190. [Online]. Available: https://ieeexplore.ieee.org/document/10254956

[5] ——, "Process-based efficient power level exporter," in *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. IEEE, 2024, pp. 456–467.

[6] V. Gudepu, R. R. Tella, C. Centofanti, J. Santos, A. Marotta, and K. Kondepu, "Demonstrating the Energy Consumption of Radio Access Networks in Container Clouds," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*, May 2024, pp. 1–3, iSSN: 2374-9709. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10575134

[7] D. Soldani, P. Nahi, H. Bour, S. Jafarizadeh, M. F. Soliman, L. Di Giovanna, F. Monaco, G. Ognibene, and F. Risso, "eBPF: A New Approach to Cloud-Native Observability, Networking and Security for Current (5G) and Future Mobile Networks (6G and Beyond)," *IEEE Access*, vol. 11, pp. 57 174–57 202, 2023, conference Name: IEEE Access. [Online]. Available: https://ieeexplore.ieee.org/document/10138542

[8] S. Choochotkaew, C. Wang, H. Chen, T. Chiba, M. Amaral, E. K. Lee, and T. Eilam, "Advancing Cloud Sustainability: A Versatile Framework for Container Power Model Training," in *2023 31st International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Oct. 2023, pp. 1–4, iSSN: 2375-0227. [Online]. Available: https://ieeexplore.ieee.org/document/10387542

[9] C. Centofanti, J. Santos, V. Gudepu, and K. Kondepu, "Impact of power consumption in containerized clouds: A comprehensive analysis of open-source power measurement tools," *Computer Networks*, vol. 245, p. 110371, May 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128624002032

[10] L. Andringa, "Estimating energy consumption of Cloud-Native applications," Master's thesis, University of Groningen, Groningen, Jul. 2024. [Online]. Available: https://fse.studenttheses.ub.rug.nl/33583/

[11] J. v. Kistowski, H. Block, J. Beckett, K.-D. Lange, J. A. Arnold, and S. Kounev, "Analysis of the Influences on Server Power Consumption and Energy Efficiency for CPU-Intensive Workloads," in *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '15. New York, NY, USA: Association for Computing Machinery, Jan. 2015, pp. 223–234. [Online]. Available: https://dl.acm.org/doi/10.1145/2668930.2688057

[12] G. Fieni, R. Rouvoy, and L. Seinturier, "SmartWatts: Self-Calibrating Software-Defined Power Meter for Containers," Jan. 2020, arXiv:2001.02505 [cs]. [Online]. Available: http://arxiv.org/abs/2001.02505

[13] G. Fieni, R. Rouvoy, and L. Seiturier, "SelfWatts: On-the-fly Selection of Performance Events to Optimize Software-defined Power Meters," in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, May 2021, pp. 324–333.

[14] M. Dinga, I. Malavolta, L. Giamattei, A. Guerriero, and R. Pietrantuono, "An Empirical Evaluation of the Energy and Performance Overhead of Monitoring Tools on Docker-Based Systems: 21st International Conference on Service-Oriented Computing, ICSOC 2023," *Service-Oriented Computing*, vol. 1, pp. 181–196, 2023.

[15] E. A. Santos, C. McLean, C. Solinas, and A. Hindle, "How does docker affect energy consumption? Evaluating workloads in and out of Docker containers," *Journal of Systems and Software*, vol. 146, pp. 14–25, Dec. 2018.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 02:16:35 UTC from IEEE Xplore. Restrictions apply.